AI Website Fabricator

Jury Code Flow Brief

Purpose: give short, defendable, code-backed answers about what the project is doing end to end.

Source of truth: current codebase in this workspace, not older milestone assumptions.

---

1. System In One Sentence

This project clones a real website into a local multi-page bundle, saves that clone, converts the clone into structured CMS data, exposes APIs and a separate admin panel for managing that data, and can finally package the whole system as a standalone connected export.

Main code references:

- ‘app.py:1665-2159‘
- ‘cloner_engine.py:1529-1717‘
- ‘smart_conversion.py:1522-1585‘
- ‘dragdrop_flask.py:621-765‘
- ‘connected_export.py:1117-1218‘

---

2. Real End-to-End Coding Flow

Step 1: the app initializes persistence

The main Streamlit app creates the project database and all main tables in ‘init_db()‘. This includes:

- ‘users‘
- ‘cloned_sites‘
- ‘conversion_blueprints‘
- ‘site_specs‘
- ‘dynamic_models‘
- ‘dynamic_records‘
- ‘media_assets‘

References:

- ‘app.py:244-379‘

Step 2: the user clones a website from Streamlit

In the ‘Clone New Site‘ menu, Streamlit accepts the URL, lets the user choose ‘Deep Clone‘ or ‘Fast Clone‘, then calls ‘scrape_website()‘, which forwards to ‘clone_site_bundle()‘ using the selected crawl limits.

References:

- ‘app.py:1533-1544‘
- ‘app.py:1705-1754‘

Step 3: the clone engine crawls the site

‘crawl_site_pages()‘ performs the crawl. It:

- normalizes the root URL
- uses a queue for BFS-style traversal
- discovers same-domain pages from links and sitemap candidates
- stops at ‘max_pages‘ and ‘max_depth‘
- stores raw HTML per discovered page

References:

- ‘cloner_engine.py:615-699‘

Step 4: the clone engine downloads and rewrites assets

After crawling, ‘clone_site_bundle()‘:

- builds local output paths for every page
- prefetches page asset requests
- rewrites each page through ‘rewrite_page_html()‘
- clones images, CSS, JS, fonts, and other assets
- rewrites links so the clone works locally
- injects a navigation bridge for cloned interactivity
- writes a manifest and a ZIP archive

References:

- ‘cloner_engine.py:1529-1717‘
- ‘cloner_engine.py:1348-1468‘
- ‘cloner_engine.py:1286-1346‘
- ‘cloner_engine.py:1471-1488‘

Important viva point:
This is not screenshot-based regeneration. It is deterministic cloning plus path rewriting.

Step 5: the staged clone is previewed before saving

The freshly cloned bundle is previewed through Flask under ‘/staged-clone/<clone_key>/...‘, and Streamlit embeds that preview in an iframe.

References:

- ‘dragdrop_flask.py:621-629‘
- ‘app.py:1809-1816‘

Step 6: the clone is saved as a project

When the user clicks ‘Save to Project Database‘, the system:

1. stores the original entry HTML in ‘cloned_sites‘
2. copies the whole staged clone into ‘project_assets/clone_packages/site_<id>/‘

References:

- ‘app.py:1794-1807‘
- ‘app.py:428-446‘

- ‘cloner_engine.py:1737-1742‘

Step 7: the saved clone can be edited in two ways

There are two editing paths:

1. Raw HTML editor in Streamlit
2. Drag-and-drop builder hosted by Flask

When HTML is updated, the app also updates the saved clone entry file and rebuilds the ZIP so the saved project stays consistent.

References:

- ‘app.py:1818-1871‘
- ‘app.py:459-466‘
- ‘cloner_engine.py:1852-1863‘
- ‘app.py:2133-2153‘
- ‘dragdrop_flask.py:541-567‘

Step 8: Smart Dynamic Conversion turns the clone into CMS data

When the user clicks ‘Generate Database + Admin Panel‘, Streamlit calls ‘generate_clone_runtime()‘. This is the key orchestration function for the dynamic side.

It does five major things:

1. builds the conversion blueprint
2. applies clone runtime context
3. writes generated artifact files
4. saves ‘SiteSpec‘ and blueprint into SQLite
5. rebuilds models, records, and media tables

References:

- ‘app.py:1931-1945‘
- ‘app.py:874-896‘

Step 9: the conversion engine analyzes the clone

‘build_conversion_blueprint()‘ is the main algorithmic entry point of the inference engine. It:

1. detects the site type
2. extracts media and localizes it
3. injects the frontend bridge
4. extracts navigation
5. extracts forms
6. extracts sections
7. infers content models
8. builds seed records
9. calculates confidence
10. builds backend/admin/frontend metadata

References:

- ‘smart_conversion.py:1522-1585‘

Step 10: site type detection is rules-based

‘detect_site_type()‘ uses keyword scoring from page text, forms, hostnames, and special keywords such as ‘shop‘, ‘university‘, ‘book‘, and ‘newsletter‘. It returns both a type and a confidence.

This means the current system uses rule-based inference, not an external ML classifier.

References:

- ‘smart_conversion.py:950-982‘

Step 11: media extraction and localization build the media library

‘extract_media_assets()‘ scans:

- image and media tags
- lazy-loading attributes
- inline styles
- style tags
- linked stylesheets
- inline SVG

Then ‘materialize_media_asset()‘ tries to download each asset into ‘project_assets/media/site_<id>/‘. If downloading fails, the system keeps it as an external reference instead of crashing.

References:

- ‘smart_conversion.py:425-645‘
- ‘smart_conversion.py:648-710‘
- ‘smart_conversion.py:712-756‘

Step 12: content models and seed records are inferred

‘extract_content_models()‘ chooses business models based on site type, then always adds system models:

- ‘site_pages‘
- ‘site_sections‘
- ‘navigation_items‘
- ‘theme_styles‘

‘extract_seed_records()‘ then fills those models using headings, paragraphs, prices, media URLs, section selectors, and navigation items extracted from the clone.

References:

- ‘smart_conversion.py:1281-1318‘
- ‘smart_conversion.py:1321-1368‘

Step 13: the confidence score is computed from multiple signals

The final confidence is not arbitrary. It is a weighted score using:

- DOM structure
- text extraction completeness
- section detection
- style match
- layout match

- media recovery
- link extraction
- interactive component detection
- site type confidence

References:

- ‘smart_conversion.py:1433-1487‘

Step 14: SiteSpec is the contract used by the admin app

‘build_site_spec()‘ converts the blueprint into a schema-driven contract containing:

- project identity
- site type
- confidence
- admin theme
- media summary
- resources and fields
- frontend bindings
- pipeline steps
- diagnostics

The React admin does not hardcode resource screens per site. It reads the generated ‘SiteSpec‘.

References:

- ‘smart_conversion.py:1497-1519‘
- ‘app.py:927-950‘

Step 15: SQLite is rebuilt with models and records

‘rebuild_dynamic_site_data()‘ deletes existing dynamic models and records for the site, then inserts:

- one row per inferred model into ‘dynamic_models‘
- one row per seed record into ‘dynamic_records‘

Seeded records are initially inserted as ‘published‘.

References:

- ‘app.py:991-1043‘

Step 16: media is synchronized into ‘media_assets‘

‘sync_media_assets()‘ upserts media rows, preserves customized media when appropriate, and archives stale clone-sourced media that is no longer present in the latest conversion output.

References:

- ‘app.py:785-872‘

Step 17: Flask becomes the service layer

‘dragdrop_flask.py‘ serves:

- staged clone previews

- saved clone previews
- clone ZIP downloads
- visual editor pages
- the built React admin
- admin APIs
- public APIs
- dynamic APIs
- media files

References:

- ‘dragdrop_flask.py:541-670‘
- ‘dragdrop_flask.py:673-1251‘

Step 18: the separate admin panel is route-driven and SiteSpec-driven

The React admin routes are defined in ‘App.tsx‘. The main protected area is:

- ‘/admin/site/:projectId‘
- ‘/admin/site/:projectId/resources/:resourceKey‘
- ‘/admin/site/:projectId/resources/:resourceKey/new‘
- ‘/admin/site/:projectId/resources/:resourceKey/:recordId‘
- ‘/admin/site/:projectId/media‘

References:

- ‘admin/src/App.tsx:91-139‘

‘AdminShell‘ loads the project spec, applies the selected admin theme, separates content resources from system resources, and builds sidebar navigation directly from ‘SiteSpec‘.

References:

- ‘admin/src/components/AdminShell.tsx:65-140‘
- ‘admin/src/components/AdminShell.tsx:195-243‘

Step 19: the admin pages perform actual CRUD

The React admin works like this:

- ‘ProjectDetailPage‘ shows overview, confidence, resources, and diagnostics
- ‘ResourceListPage‘ lists records and supports publish/delete
- ‘RecordEditorPage‘ renders fields from ‘SiteSpec‘
- ‘MediaPage‘ handles upload, replace, delete, preview, and search

References:

- ‘admin/src/pages/ProjectDetailPage.tsx:8-227‘
- ‘admin/src/pages/ResourceListPage.tsx:64-240‘
- ‘admin/src/pages/RecordEditorPage.tsx:49-279‘
- ‘admin/src/pages/MediaPage.tsx:50-260‘

On the backend side, CRUD is implemented through:

- ‘/api/admin/<site_id>/<resource_key>‘
- ‘/api/admin/<site_id>/<resource_key>/<record_id>‘
- ‘/api/admin/<site_id>/<resource_key>/<record_id>/publish‘

- ‘/api/admin/<site_id>/media‘
- ‘/api/admin/<site_id>/media/<media_id>‘

References:

- ‘dragdrop_flask.py:841-977‘
- ‘dragdrop_flask.py:980-1165‘

Step 20: the public side reads published data only

The clone bridge and external consumers use published/public endpoints. The public resource endpoint filters records by ‘status = ’published’‘.

References:

- ‘dragdrop_flask.py:1174-1187‘

Step 21: connected export creates a standalone package

‘build_connected_export()‘ creates a full delivery package containing:

- ‘cloned_website/‘
- ‘admin_system/admin_panel/‘
- ‘admin_system/backend/‘
- ‘admin_system/database/site.db‘
- ‘admin_system/media/‘
- ‘admin_system/generated_dynamic/‘

It copies only the selected site?s database subset, injects the clone bridge into exported HTML, writes a standalone Flask backend, writes startup scripts, and creates a final ZIP.

References:

- ‘connected_export.py:1117-1218‘
- ‘connected_export.py:129-193‘
- ‘connected_export.py:1024-1055‘
- ‘connected_export.py:1056-1114‘

---

3. File-by-File Technical Role

‘app.py‘

Role:

- Streamlit control center
- database initialization
- login/registration
- clone trigger
- saved project manager
- raw HTML editor
- Smart Dynamic Conversion trigger
- export trigger

Best defense line:

‘app.py‘ is the orchestration layer. It does not do the heavy crawling or inference itself; it coordinates other modules and persists the results.

Key references:

- ‘app.py:244-379‘
- ‘app.py:1665-2159‘

‘cloner_engine.py‘

Role:

- crawl the target site
- discover internal pages
- download assets
- rewrite HTML/CSS/JS references
- inject a navigation bridge
- build a local multi-page bundle and ZIP

Best defense line:
This file is the deterministic website cloning engine.

Key references:

- ‘cloner_engine.py:615-699‘
- ‘cloner_engine.py:1348-1468‘
- ‘cloner_engine.py:1529-1717‘

‘smart_conversion.py‘

Role:

- infer site type
- extract navigation, sections, forms, and media
- localize media
- infer content models
- create seed records
- compute confidence
- create ‘SiteSpec‘

Best defense line:
This file transforms a static clone into structured, manageable CMS data.

Key references:

- ‘smart_conversion.py:425-756‘
- ‘smart_conversion.py:950-1585‘

‘dragdrop_flask.py‘

Role:

- serves clone previews and static assets
- hosts the drag-and-drop editor
- hosts the built React admin
- exposes admin/public/dynamic APIs

Best defense line:
This file is the shared runtime service layer for editing, previewing, and managing data.

Key references:

- ‘dragdrop_flask.py:526-670‘
- ‘dragdrop_flask.py:673-1251‘

‘connected_export.py‘

Role:

- creates a standalone package for one site
- exports a site-scoped database
- writes a separate backend runtime
- writes run scripts and manifests

Best defense line:
This file turns the project from a local development tool into a distributable connected system.

Key references:

- ‘connected_export.py:53-126‘
- ‘connected_export.py:129-193‘
- ‘connected_export.py:1024-1218‘

‘admin/src/*‘

Role:

- React admin frontend
- uses ‘SiteSpec‘ to generate navigation and forms
- performs CRUD over Flask APIs
- gives a non-technical CMS-style interface for one cloned site

Key references:

- ‘admin/src/App.tsx:91-139‘
- ‘admin/src/components/AdminShell.tsx:65-140‘
- ‘admin/src/pages/ProjectDetailPage.tsx:8-227‘
- ‘admin/src/pages/ResourceListPage.tsx:64-240‘
- ‘admin/src/pages/RecordEditorPage.tsx:49-279‘
- ‘admin/src/pages/MediaPage.tsx:50-260‘

---

4. Core Algorithms You Should Be Ready To Explain

Clone algorithm

Short answer:
The system performs controlled same-domain crawling, then rewrites every captured page and asset reference into a local bundle.

References:

- ‘cloner_engine.py:615-699‘

- ‘cloner_engine.py:1529-1717‘

Path rewriting algorithm

Short answer:
For each HTML page, the engine resolves each link or asset URL, decides whether it should remain a page link or become a downloaded asset, then rewrites the attribute to the new relative local path.

References:

- ‘cloner_engine.py:1348-1468‘

Site type detection algorithm

Short answer:
The system scores categories such as ‘ecommerce‘, ‘education‘, ‘blog‘, and ‘service_business‘ using keyword and hostname rules, then picks the highest-scoring type.

References:

- ‘smart_conversion.py:950-982‘

Content model inference algorithm

Short answer:
The detected site type selects a base set of business collections, then the system always adds internal system collections such as pages, sections, navigation items, and theme styles.

References:

- ‘smart_conversion.py:1281-1318‘

Seed record generation algorithm

Short answer:
The system uses extracted headings, paragraphs, prices, navigation items, sections, and media URLs to synthesize initial records that match the inferred schema.

References:

- ‘smart_conversion.py:1321-1368‘

Confidence algorithm

Short answer:
The confidence is a weighted score across structure, text, sections, styles, layout, media recovery, link coverage, interactivity, and site-type certainty.

References:

- ‘smart_conversion.py:1433-1487‘

---

5. Database Story In Simple Words

‘users‘ stores who owns the work.

‘cloned_sites‘ stores the saved clone entry HTML and source URL.

‘conversion_blueprints‘ stores the full inferred analysis result.

‘site_specs‘ stores the schema contract used by the admin app.

‘dynamic_models‘ stores the inferred collections.

‘dynamic_records‘ stores actual content rows.

‘media_assets‘ stores localized or uploaded media and mapping metadata.

References:

- ‘app.py:244-379‘

---

6. Diagram Explanations For Viva

High-Level System Flow

Use this line:
This diagram shows the life of one website from URL input to clone bundle, structured conversion, admin management, public API delivery, and final export packaging.

System Architecture

Use this line:
This diagram separates the project into five layers: Streamlit control, clone and inference logic, persistence, Flask services, and the React admin UI.

Use Case Diagram

Use this line:
This diagram shows that the technical operator handles cloning and generation, while the content manager only manages structured content inside the site-scoped admin.

Class / Module Diagram

Use this line:
This diagram shows which core modules exist in code and how orchestration, cloning, conversion, serving, admin UI, export, and database access connect together.

End-to-End Sequence Diagram

Use this line:
This diagram shows the exact runtime order: clone, stage preview, save project, convert, open admin, manage records, and export.

Data Flow Diagram

Use this line:

This diagram focuses on how data changes form, especially how HTML becomes SiteSpec, models, records, and media assets.

Entity Relationship Diagram

Use this line:
This diagram explains the seven-table schema and how one cloned site owns its blueprint, spec, models, records, and media.

Deployment and Runtime Diagram

Use this line:
This diagram shows the real runtime processes and ports used during development and in the exported standalone system.

Connected Export Package Structure

Use this line:
This diagram explains the final packaged system, where the clone, admin panel, backend, database, media, and generated artifacts are bundled together.

---

7. Likely Jury Questions With Direct Answers

What is the main entry point?

‘app.py‘ is the main user-facing entry point because it runs the Streamlit control app and triggers the rest of the pipeline.

Reference:

- ‘app.py:1665-2159‘

Where does cloning actually happen?

In ‘cloner_engine.py‘, mainly through ‘clone_site_bundle()‘, ‘crawl_site_pages()‘, and ‘rewrite_page_html()‘.

References:

- ‘cloner_engine.py:615-699‘
- ‘cloner_engine.py:1348-1468‘
- ‘cloner_engine.py:1529-1717‘

Where does the CMS schema come from?

From ‘smart_conversion.py‘, especially ‘extract_content_models()‘ and ‘build_site_spec()‘.

References:

- ‘smart_conversion.py:1281-1318‘
- ‘smart_conversion.py:1497-1519‘

How does the admin know what screens to show?

The admin reads the generated ‘SiteSpec‘, then builds navigation and forms from ‘spec.resources‘.

References:

- ‘smart_conversion.py:1497-1519‘
- ‘admin/src/components/AdminShell.tsx:130-143‘

How is the clone updated when content changes?

Through the frontend bridge and public APIs. The clone receives the injected bridge script and can fetch content from the public endpoints.

References:

- ‘smart_conversion.py:759-820‘
- ‘dragdrop_flask.py:1174-1187‘

Why is Flask needed if Streamlit already exists?

Streamlit is the operator dashboard. Flask is the service/runtime layer for previews, APIs, drag-and-drop editing, media serving, and admin hosting.

References:

- ‘app.py:1665-2159‘
- ‘dragdrop_flask.py:526-670‘

What makes the export standalone?

The export contains its own backend, site database, built admin, cloned frontend, media, and generated artifacts.

References:

- ‘connected_export.py:1117-1218‘

---

8. Final Takeaway

If the jury asks what this project really is, the best short answer is:

AI Website Fabricator is a clone-first website re-engineering system. It captures a live website deterministically, converts that clone into structured CMS data, serves a separate admin panel for managing the content, and can package the result as a standalone connected system.